

Low-rank Connectivity and the Dynamics It Produces

A short note on the framework used by Dubreuil et al. (2022)

1 The starting point: a recurrent neural network

The paper studies firing-rate recurrent neural networks. A common continuous-time form is

$$\tau \frac{dx_i}{dt} = -x_i + \sum_{j=1}^N J_{ij} \phi(x_j) + \sum_{s=1}^{N_{\text{in}}} I_i^{(s)} u_s(t), \quad (1)$$

where:

- $x_i(t)$ is the activation of neuron i ;
- $\phi(x_i)$ is its firing rate after a static nonlinearity;
- J_{ij} is the recurrent weight from neuron j to neuron i ;
- $u_s(t)$ are external inputs, such as stimulus or context signals;
- $I_i^{(s)}$ is the input weight from input channel s to neuron i .

In vector form,

$$\tau \dot{\mathbf{x}} = -\mathbf{x} + J\phi(\mathbf{x}) + \sum_s \mathbf{I}^{(s)} u_s(t). \quad (2)$$

A completely general $N \times N$ recurrent matrix J can have rank as large as N . Such a matrix can create many independent recurrent feedback directions. The low-rank assumption sharply restricts this.

2 What “low rank” means

A rank- R recurrent connectivity matrix can be written as a sum of R outer products:

$$J = \frac{1}{N} \sum_{r=1}^R \mathbf{m}^{(r)} \left(\mathbf{n}^{(r)} \right)^\top, \quad (3)$$

or componentwise,

$$J_{ij} = \frac{1}{N} \sum_{r=1}^R m_i^{(r)} n_j^{(r)}. \quad (4)$$

Each term has two population-length vectors:

- $\mathbf{n}^{(r)}$ is a *read-in* or *selection* vector. It defines which population activity pattern is measured.
- $\mathbf{m}^{(r)}$ is a *write-out* vector. It defines which pattern of recurrent input is sent back into the neurons.

For rank one,

$$J = \frac{1}{N} \mathbf{m} \mathbf{n}^\top, \quad (5)$$

so recurrent processing has a simple form:

$$J\phi(\mathbf{x}) = \mathbf{m} \left(\frac{1}{N} \mathbf{n}^\top \phi(\mathbf{x}) \right). \quad (6)$$

The network first projects the current firing rates onto $\mathbf{n}$, producing one scalar, and then feeds that scalar back along $\mathbf{m}$.

3 The latent variable

The scalar produced by the projection is the key low-dimensional variable:

$$\kappa(t) = \frac{1}{N} \sum_{j=1}^N n_j \phi(x_j(t)). \quad (7)$$

For rank one, the recurrent input to neuron i is simply

$$m_i \kappa(t). \quad (8)$$

Thus the full N -dimensional network is not using an arbitrary recurrent input pattern. Its recurrent input always lies along the single population direction $\mathbf{m}$. For rank R , there are R such latent variables:

$$\kappa_r(t) = \frac{1}{N} \sum_{j=1}^N n_j^{(r)} \phi(x_j(t)), \quad r = 1, \dots, R. \quad (9)$$

The recurrent input is then

$$J\phi(\mathbf{x}) = \sum_{r=1}^R \mathbf{m}^{(r)} \kappa_r(t). \quad (10)$$

This is the main mathematical reason low-rank networks are interpretable: the large neural population can be described through a small set of recurrent latent variables $\kappa_1, \dots, \kappa_R$.

4 Why low-rank connectivity creates low-dimensional dynamics

Insert the low-rank form into the RNN:

$$\tau \dot{\mathbf{x}} = -\mathbf{x} + \sum_{r=1}^R \mathbf{m}^{(r)} \kappa_r(t) + \sum_s \mathbf{I}^{(s)} u_s(t). \quad (11)$$

This equation says that changes in the neural state are built only from:

- the recurrent directions $\mathbf{m}^{(1)}, \dots, \mathbf{m}^{(R)}$;
- the input directions $\mathbf{I}^{(1)}, \dots, \mathbf{I}^{(N_{\text{in}})}$;
- the leak term $-\mathbf{x}$.

If the initial state is in the subspace spanned by the recurrent and input directions, it stays there. Even when the full network has thousands of neurons, the meaningful trajectories can live in a much smaller subspace. This is what the paper means by reducing the network to latent dynamics.

5 A rank-one example

With one input $u(t)$ and rank-one recurrent connectivity,

$$\tau \dot{x}_i = -x_i + m_i \kappa(t) + I_i u(t), \quad (12)$$

$$\kappa(t) = \frac{1}{N} \sum_j n_j \phi(x_j(t)). \quad (13)$$

If ϕ were linear, the latent dynamics would be approximately

$$\tau \dot{\kappa} = -\kappa + \sigma_{nm} \kappa + \sigma_{nI} u(t), \quad (14)$$

where

$$\sigma_{nm} = \frac{1}{N} \sum_i n_i m_i, \quad \sigma_{nI} = \frac{1}{N} \sum_i n_i I_i. \quad (15)$$

These overlaps control the computation:

- σ_{nI} says how strongly the input drives the latent variable;
- σ_{nm} says how strongly the latent variable feeds back onto itself.

If feedback is weak, κ tracks the input briefly. If feedback is tuned close to the leak, κ can integrate information over time. If the nonlinearity is strong and feedback is positive, the same low-dimensional system can support attractor-like decision states.

6 Where population structure enters

The paper then asks: what statistics of the neuron-specific parameters are needed for a task? For each neuron, collect its coordinates in connectivity space:

$$\left(m_i^{(1)}, \dots, m_i^{(R)}, n_i^{(1)}, \dots, n_i^{(R)}, I_i^{(1)}, \dots, I_i^{(N_{\text{in}})}, w_i \right). \quad (16)$$

Across neurons, these coordinates form a distribution.

Single random population

In a fully random population structure, these neuron coordinates are modeled as draws from one joint distribution, often a multivariate Gaussian:

$$\mathbf{a}_i \sim \mathcal{N}(0, \Sigma). \quad (17)$$

The effective latent dynamics are governed by global covariances such as σ_{nm} and σ_{nI} . There are no distinct groups of neurons with different computational roles.

Multiple subpopulations

In a non-random population structure, the coordinates are better described as a mixture:

$$p(\mathbf{a}) = \sum_{p=1}^P \alpha_p \mathcal{N}(\mu_p, \Sigma_p). \quad (18)$$

Each component corresponds to a subpopulation. The subpopulations can have different covariances, so the same input may modulate one group strongly and another weakly.

In the nonlinear case, this is crucial. Dubreuil et al. write the effective couplings as population-weighted quantities of the form

$$\tilde{\sigma}_{ab} = \sum_p \alpha_p \sigma_{ab}^{(p)} \langle \phi' \rangle_p. \quad (19)$$

Here $\langle \phi' \rangle_p$ is the gain of subpopulation p . Inputs can change these gains differently across subpopulations. This lets context reshape the effective latent dynamics without necessarily driving the same latent variable directly.

7 The computational message

Low rank controls the *dimension* of the recurrent dynamics. A rank- R network has R main recurrent latent variables.

Population structure controls how inputs, feedback, and readout are arranged across neurons. A single random population can implement many tasks by using low-dimensional dynamics. But tasks requiring flexible input-output remapping can require multiple subpopulations, because different groups need to have different gain modulations and different effective roles.

In one sentence:

Low-rank connectivity says the network computes through a small number of collective variables; non-random population structure says different neurons are organized so that inputs can flexibly reshape the dynamics of those collective variables.

Glossary

Term	Meaning
J	recurrent connectivity matrix
rank R	number of independent recurrent feedback directions
$\mathbf{n}^{(r)}$	vector that reads population activity into latent variable r
$\mathbf{m}^{(r)}$	vector that writes latent variable r back into the population
κ_r	low-dimensional recurrent latent variable
$\mathbf{I}^{(s)}$	input vector for input channel s
$\mathbf{w}$	readout vector for producing the network output
population structure	distribution of neuron coordinates in connectivity space

8 Additional primer for implementing the assignment

The original note above explains why low-rank connectivity is useful. The remaining sections add the implementation details needed to build and train the specific rank-one RNN from the assignment.

9 The concrete model in the task

The task asks for a recurrent neural network with one-dimensional input $u(t) \in \mathbb{R}$, one-dimensional output $z(t) \in \mathbb{R}$, and N recurrent units. The continuous-time state of neuron i is $x_i(t)$. Its firing rate is

$$r_i(t) = \phi(x_i(t)) = \tanh(x_i(t)). \quad (20)$$

The dynamics are

$$\tau \frac{dx_i}{dt} = -x_i + \sum_{j=1}^N J_{ij} \phi(x_j) + I_i u(t), \quad i = 1, \dots, N, \quad (21)$$

with neuronal time constant $\tau = 100$ ms.

The recurrent matrix is rank one:

$$J = \frac{1}{N} \mathbf{m} \mathbf{n}^\top, \quad J_{ij} = \frac{1}{N} m_i n_j. \quad (22)$$

The output is a linear readout of the rates:

$$z(t) = \frac{1}{N} \sum_{i=1}^N w_i \phi(x_i(t)). \quad (23)$$

Only $\mathbf{m}$ and $\mathbf{n}$ are trainable. The input vector $\mathbf{I}$ and output vector $\mathbf{w}$ are fixed after initialization. All trainable and fixed parameters are sampled from standard normal distributions, except $\mathbf{w}$, which is sampled with standard deviation 4.

9.1 Vector form

Let $\mathbf{x}(t) \in \mathbb{R}^N$, $\mathbf{I}, \mathbf{m}, \mathbf{n}, \mathbf{w} \in \mathbb{R}^N$, and $\phi(\mathbf{x})$ mean applying $\tanh$ elementwise. Then

$$\tau \dot{\mathbf{x}} = -\mathbf{x} + \frac{1}{N} \mathbf{m} \mathbf{n}^\top \phi(\mathbf{x}) + \mathbf{I} u(t). \quad (24)$$

Because J is rank one, it is better not to explicitly build the full $N \times N$ matrix. Define the scalar recurrent coordinate

$$\kappa(t) = \frac{1}{N} \mathbf{n}^\top \phi(\mathbf{x}(t)). \quad (25)$$

Then the recurrent input is simply

$$J \phi(\mathbf{x}(t)) = \mathbf{m} \kappa(t). \quad (26)$$

So the implemented update can use one dot product with $\mathbf{n}$, then multiply by $\mathbf{m}$. This is both clearer and cheaper than forming J .

10 What an RNN is doing

A recurrent neural network is a dynamical system with memory. A feedforward network computes

$$y = f_\theta(u) \quad (27)$$

from the current input alone. An RNN computes from both the current input and an internal state:

$$\mathbf{x}_{t+1} = F_\theta(\mathbf{x}_t, u_t), \quad z_t = G_\theta(\mathbf{x}_t). \quad (28)$$

The state $\mathbf{x}_t$ carries information from earlier inputs. In this task, the state is a population of firing-rate units. The leak term $-\mathbf{x}$ makes old activity decay, the recurrent term feeds current activity back into the network, and the input term injects the stimulus.

The nonlinearity matters. Here $\phi = \tanh$ maps activations to rates in $(-1, 1)$. Around zero, $\tanh(x) \approx x$, so the network behaves almost linearly. For large positive or negative activations, $\tanh$ saturates and $\phi'(x) = 1 - \tanh^2(x)$ becomes small. Saturation can stabilize activity, but it can also make gradients small during training.

11 From the differential equation to code

The task specifies forward Euler simulation with

$$\Delta t = 20 \text{ ms.}$$

Since $\tau = 100 \text{ ms}$,

$$\alpha = \frac{\Delta t}{\tau} = \frac{20}{100} = 0.2.$$

Forward Euler replaces $\dot{\mathbf{x}}(t)$ by

$$\frac{\mathbf{x}_{t+1} - \mathbf{x}_t}{\Delta t}.$$

Substituting into Equation 24 gives

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \alpha [-\mathbf{x}_t + \mathbf{m}\kappa_t + \mathbf{I}u_t], \quad (29)$$

where

$$\kappa_t = \frac{1}{N} \mathbf{n}^\top \tanh(\mathbf{x}_t). \quad (30)$$

Equivalently,

$$\mathbf{x}_{t+1} = (1 - \alpha)\mathbf{x}_t + \alpha\mathbf{m}\kappa_t + \alpha\mathbf{I}u_t. \quad (31)$$

This is the exact recurrence to implement.

If the stimulus lasts 75 time steps, the real duration is

$$75 \times 20 \text{ ms} = 1500 \text{ ms.}$$

12 Tensor shapes for a batch implementation

Use a batch dimension so many trials can be simulated at once. One convenient layout is:

Quantity	Shape	Meaning
u	(batch, time)	scalar input sequence for each trial
x	(batch, N)	current recurrent state
rates	(batch, N)	$\tanh(\mathbf{x})$
m, n, I, w	(N,)	population vectors
kappa	(batch,)	recurrent latent coordinate
z	(batch,)	scalar output at one time step
outputs	(batch, time)	full output sequence

With this layout, the low-rank computations are:

$$\kappa_t^{(b)} = \frac{1}{N} \sum_{i=1}^N n_i \tanh(x_{t,i}^{(b)}), \quad (32)$$

$$z_t^{(b)} = \frac{1}{N} \sum_{i=1}^N w_i \tanh(x_{t,i}^{(b)}), \quad (33)$$

where b indexes the trial in the batch.

In PyTorch, these become:

```
rates = torch.tanh(x)           # (batch, N)
kappa = (rates * n).mean(dim=1)  # (batch,)
z = (rates * w).mean(dim=1)      # (batch,)
```

The use of `mean(dim=1)` implements the factor $1/N$.

13 Initialization and the role of the output scale

The vectors should be initialized as

$$m_i \sim \mathcal{N}(0, 1), \quad n_i \sim \mathcal{N}(0, 1), \quad I_i \sim \mathcal{N}(0, 1), \quad w_i \sim \mathcal{N}(0, 4^2). \quad (34)$$

The larger standard deviation for $\mathbf{w}$ compensates for the averaging in the readout. At random initialization, if the rates are only weakly aligned with $\mathbf{w}$, the sum

$$\frac{1}{N} \sum_i w_i r_i$$

has a typical scale proportional to $\text{std}(w)/\sqrt{N}$ rather than $\text{std}(w)$. A larger output scale helps keep the initial output from being extremely small for moderate N . It does not make $\mathbf{w}$ trainable; it only sets the fixed readout scale.

14 Trainable parameters versus fixed buffers

In PyTorch, trainable parameters should be `nn.Parameter` objects. Fixed tensors that belong to the model should be registered as buffers. Buffers move with the model when calling `model.to(device)`, are saved in the `state_dict`, but are not updated by the optimizer.

For this task:

Vector	PyTorch representation	Optimizer updates it?
m	<code>nn.Parameter</code>	yes
n	<code>nn.Parameter</code>	yes
I	<code>register_buffer("I", ...)</code>	no
w	<code>register_buffer("w", ...)</code>	no

15 A minimal PyTorch module

This is the core implementation pattern. It avoids forming J and unrolls the dynamics over time.

```
import torch
from torch import nn

class RankOneRNN(nn.Module):
    def __init__(self, n_units: int, dt_ms: float = 20.0, tau_ms: float = 100.0):
        super().__init__()
        self.n_units = n_units
        self.alpha = dt_ms / tau_ms

        self.m = nn.Parameter(torch.randn(n_units))
        self.n = nn.Parameter(torch.randn(n_units))

        self.register_buffer("I", torch.randn(n_units))
        self.register_buffer("w", 4.0 * torch.randn(n_units))

    def forward(self, u: torch.Tensor, x0: torch.Tensor | None = None):
        # u has shape (batch, time)
        batch_size, n_steps = u.shape

        if x0 is None:
            x = u.new_zeros(batch_size, self.n_units)
        else:
            x = x0

        outputs = []
        states = []

        for t in range(n_steps):
            rates = torch.tanh(x)

            z = (rates * self.w).mean(dim=1)
            outputs.append(z)
```



```

        states.append(x)

        kappa = (rates * self.n).mean(dim=1)
        recurrent = kappa[:, None] * self.m[None, :]
        external = u[:, t, None] * self.I[None, :]

        x = x + self.alpha * (-x + recurrent + external)

    outputs = torch.stack(outputs, dim=1)
    states = torch.stack(states, dim=1)
    return outputs, states

```

Important shape details:

- `kappa` has shape `(batch,)`.
- `kappa[:, None]` has shape `(batch, 1)`.
- `self.m[None, :]` has shape `(1, N)`.
- Their product broadcasts to `(batch, N)`.

16 Training objective

Training means choosing $\mathbf{m}$ and $\mathbf{n}$ so the output sequence z_t matches a target sequence y_t . A standard objective is mean squared error:

$$\mathcal{L}(\mathbf{m}, \mathbf{n}) = \frac{1}{BT} \sum_{b=1}^B \sum_{t=1}^T \left(z_t^{(b)} - y_t^{(b)} \right)^2, \quad (35)$$

where B is batch size and T is the number of time steps.

Often the loss is computed only over task-relevant time points. If the network is only supposed to report a decision after the stimulus or during a response window, use a mask:

$$\mathcal{L} = \frac{\sum_{b,t} M_t \left(z_t^{(b)} - y_t^{(b)} \right)^2}{\sum_t M_t B}. \quad (36)$$

Here $M_t \in \{0, 1\}$ selects the time steps that matter.

In code:

```

outputs, states = model(u)                # outputs: (batch, time)
sq_error = (outputs - target) ** 2

if mask is not None:                      # mask: (time,) or (batch, time)
    loss = (sq_error * mask).sum() / mask.expand_as(sq_error).sum()
else:
    loss = sq_error.mean()

```

17 How training works: backpropagation through time

The forward pass unrolls the recurrence:

$$\mathbf{x}_1 = F_\theta(\mathbf{x}_0, u_0), \quad (37)$$

$$\mathbf{x}_2 = F_\theta(\mathbf{x}_1, u_1), \quad (38)$$

$$\vdots \quad (39)$$

$$\mathbf{x}_T = F_\theta(\mathbf{x}_{T-1}, u_{T-1}), \quad (40)$$

where $\theta = \{\mathbf{m}, \mathbf{n}\}$ in this task.

The loss depends on all states through the outputs:

$$\mathcal{L} = \sum_t \ell(G(\mathbf{x}_t), y_t).$$

Backpropagation through time is ordinary backpropagation applied to this unrolled computation graph. The same parameters $\mathbf{m}$ and $\mathbf{n}$ are reused at every time step, so their gradients accumulate contributions from every step:

$$\frac{\partial \mathcal{L}}{\partial \theta} = \sum_{t=1}^T \frac{\partial \mathcal{L}}{\partial \mathbf{x}_t} \frac{\partial \mathbf{x}_t}{\partial \theta}.$$

The dependence of later states on earlier states introduces products of Jacobians:

$$\frac{\partial \mathbf{x}_t}{\partial \mathbf{x}_s} = \prod_{q=s}^{t-1} \frac{\partial F_\theta(\mathbf{x}_q, u_q)}{\partial \mathbf{x}_q}, \quad s < t.$$

If these products shrink, gradients vanish. If they grow, gradients explode. The leak, the factor $\alpha = 0.2$, the low rank, and the saturation of tanh all affect this gradient flow.

PyTorch builds the graph automatically while the loop runs. Calling `loss.backward()` computes the gradients; calling `optimizer.step()` updates only `nn.Parameter` tensors.

```
model = RankOneRNN(n_units=256)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

for u_batch, target_batch in train_loader:
    optimizer.zero_grad()
    outputs, states = model(u_batch)
    loss = ((outputs - target_batch) ** 2).mean()
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
    optimizer.step()
```

Gradient clipping is not part of the mathematical model, but it is often useful for RNNs because repeated recurrence can create large gradients.

18 What gradients hit in this rank-one model

The trainable vectors influence the network differently.

The vector $\mathbf{n}$ determines what population activity is measured:

$$\kappa_t = \frac{1}{N} \sum_i n_i r_{t,i}.$$

Changing $\mathbf{n}$ changes the scalar feedback signal extracted from the rates.

The vector $\mathbf{m}$ determines where that scalar is written back:

$$\text{recurrent input}_{t,i} = m_i \kappa_t.$$

Changing $\mathbf{m}$ changes which neurons receive positive or negative recurrent drive.

Because $\mathbf{w}$ is fixed, training cannot directly rotate the output readout. Instead, it must shape the recurrent dynamics so that the fixed readout

$$z_t = \frac{1}{N} \mathbf{w}^\top \tanh(\mathbf{x}_t)$$

becomes task-correct.

19 Low-rank interpretation

A full recurrent matrix has N^2 entries and can mix neural activity in many directions. The rank-one matrix

$$J = \frac{1}{N} \mathbf{m} \mathbf{n}^\top$$

has only two length- N vectors. It performs two operations:

1. read the current rates along $\mathbf{n}$ to get one scalar κ ;
2. write that scalar back into the population along $\mathbf{m}$.

This makes the recurrent part of the dynamics low-dimensional:

$$\tau \dot{\mathbf{x}} = -\mathbf{x} + \mathbf{m} \kappa(t) + \mathbf{I} u(t).$$

Even though the state $\mathbf{x}$ has N components, the recurrent feedback has only one degree of freedom. The main computation can often be understood by tracking $\kappa(t)$ and the output $z(t)$.

For intuition, suppose $\tanh(x) \approx x$. Then

$$\tau \dot{\kappa} \approx -\kappa + \sigma_{nm} \kappa + \sigma_{nI} u(t),$$

where

$$\sigma_{nm} = \frac{1}{N} \sum_i n_i m_i, \quad \sigma_{nI} = \frac{1}{N} \sum_i n_i I_i.$$

The overlap σ_{nI} controls how strongly the input drives the latent coordinate. The overlap σ_{nm} controls recurrent feedback. Training $\mathbf{m}$ and $\mathbf{n}$ changes these overlaps and thereby changes whether activity decays quickly, integrates evidence, or moves toward stable decision-like states.

20 Practical implementation checks

Before training on the full task, test the model in small pieces:

1. Check that `model(u)` returns `outputs` with shape `(batch, time)` and `states` with shape `(batch, time, N)`.
2. Check that only `m` and `n` appear in `list(model.parameters())`.
3. Check that `I` and `w` appear in `model.state_dict()` but not in the optimizer parameters.
4. Feed zero input and verify that the zero initial state stays zero.
5. Feed a simple constant input and verify that states and outputs are finite, not `nan` or `inf`.
6. Run one training step and verify that `m.grad` and `n.grad` are not `None`.

Example checks:

```
u = torch.randn(8, 75)
model = RankOneRNN(n_units=128)
outputs, states = model(u)

assert outputs.shape == (8, 75)
assert states.shape == (8, 75, 128)
assert torch.isfinite(outputs).all()

loss = outputs.pow(2).mean()
loss.backward()
assert model.m.grad is not None
assert model.n.grad is not None
assert model.I.grad is None
assert model.w.grad is None
```

21 Common pitfalls

- Forgetting the factor $1/N$. In code, use `mean(dim=1)` or divide sums by `n_units`.
- Accidentally making `I` or `w` trainable. They should be buffers for this task.
- Building the full J matrix unnecessarily. For rank one, use `kappa * m`.
- Mixing time and batch dimensions. Pick one layout and keep it consistent.
- Detaching the state inside the time loop during normal full-sequence training. Detaching cuts the gradient path through time.
- Using in-place tensor operations on values needed for autograd. Prefer assignments like `x = x + ...`
- Computing the output only after the update when the target expects output before the update, or vice versa. Choose the convention deliberately and keep targets aligned with it.

22 Minimal mental model

At each time step, the network does the following:

1. Convert activations to firing rates:

$$\mathbf{r}_t = \tanh(\mathbf{x}_t).$$

2. Read the population along $\mathbf{n}$:

$$\kappa_t = \frac{1}{N} \mathbf{n}^\top \mathbf{r}_t.$$

3. Produce the output through the fixed readout:

$$z_t = \frac{1}{N} \mathbf{w}^\top \mathbf{r}_t.$$

4. Combine leak, recurrent feedback, and input:

$$\mathbf{x}_{t+1} = \mathbf{x}_t + 0.2 [-\mathbf{x}_t + \mathbf{m}\kappa_t + \mathbf{I}u_t].$$

5. During training, compare z_t to the target, backpropagate through the unrolled steps, and update only $\mathbf{m}$ and $\mathbf{n}$.

That is the core of the assignment: implement this recurrence correctly, generate outputs over time, define a task loss, and let PyTorch’s autograd tune the two low-rank connectivity vectors.

Glossary

Term	Meaning
$\mathbf{x}_t$	recurrent state or activation vector at time step t
$\tanh(\mathbf{x}_t)$	firing-rate vector after the nonlinearity
τ	neuronal time constant; here 100 ms
Δt	Euler step size; here 20 ms
α	discretization factor $\Delta t/\tau = 0.2$
J	recurrent connectivity matrix
rank one	one recurrent read-in/write-out direction pair
$\mathbf{n}$	trainable vector that reads rates into κ
$\mathbf{m}$	trainable vector that writes κ back into neurons
κ_t	scalar recurrent latent coordinate
$\mathbf{I}$	fixed input vector
$\mathbf{w}$	fixed output readout vector
z_t	scalar network output
BPTT	backpropagation through time; backprop on the unrolled recurrence